Lab 1

1. Create GPU Session
    a. Login to hipergator
    b. Allocate resources
        i. Job Number: 43658210
        ii. Node Name: c1105a-s5
    c. Step Into Node
2. Create the CPU reference functions
    a. void vecadd_cpu(const float* x, const float* y, float* z, int N);
    b. bool verify(const float* x, const float* y, float* z, int N);
    c. void add_range(const float* x, const float* y, float* z, int begin, int end);
    d. void vecadd_cpu_mt(const float* x, const float* y, float* z, int N, int T);
3. File Structure
    a. lab1/
    b. ├── README.md
    c. ├── vector_add.cu // allocates inputs, calls each version, verifies, times, prints
    d. ├── vector_add.exe // regular executable
    e. ├── vector_add_debug.exe // debug executable
    f. └── results/ // saved program output
    g. Create GPU functions
    h. Found in powerpoints and slides
4. Build and Run Configurations
    a. nvcc -O2 -lineinfo vector_add.cu -o vector_add
    b. ./vector_add 1000
    c. ./vector_add 1011 | tee "results/n1011.txt"
    d. nvcc -O0 -g -G vector_add.cu -o vector_add_debug
    e. compute-sanitizer --tool memcheck ./vector_add_debug 1000

Timing Scope and Reporting

Test Case	N Size	Timing Results
N = 1	N = 1	• Timing Results • CPU Time = 0.000050 • CPU_MT Time = 0.150608 • GPU Kernel Time = 0.006180 • GPU Application Time=3.756503
N < 1 Block	N = 115	• Timing Results • CPU Time = 0.000100 • CPU_MT Time = 0.163077 • GPU Kernel Time = 0.007061 • GPU Application Time=3.322839
N = 1 Block - 1	N = 255	• Timing Results • CPU Time = 0.000171 • CPU_MT Time = 0.172430 • GPU Kernel Time = 0.006941 • GPU Application Time=3.561669
N = 1 Block	N = 256	• Timing Results • CPU Time = 0.000150 • CPU_MT Time = 0.148004 • GPU Kernel Time = 0.007191 • GPU Application Time=2.640729
N = 1 Block + 1	N = 257	• Timing Results • CPU Time = 0.000170 • CPU_MT Time = 0.147824 • GPU Kernel Time = 0.006500 • GPU Application Time=2.131950
N not divisible by Block Size	N = 1011	• Timing Results • CPU Time = 0.000451 • CPU_MT Time = 0.162175 • GPU Kernel Time = 0.007151 • GPU Application Time=3.102896
Large N for Multiple Blocks	N = 190000	• Timing Results • CPU Time = 0.073260 • CPU_MT Time = 0.182496 • GPU Kernel Time = 0.007912 • GPU Application Time=3.095776

Results Table for Correctness Matrix

The results here also include values from the launch configuration exercise.

numThreadsPerBlock = 256

For N < 1 block, N < 256

For N = 1 block, N = 256

For N not divisible by block size, it can be any N value that isn't divisible by the number of blocks

Ex: 3 * 256 = 768, 4 * 256 = 1024, so pick any value in between

Test Case	N Size	Results
N = 1	N = 1	● N=1 ● Threads=256 ● Blocks=1 ● Threads/Block = 256 ● Tolerance = 0.000010 ● CPU_Result=5842000 ● GPU_RESULT=5841968 ● PASS/FAIL: PASS
N < 1 Block	N = 115	● N=115 ● Threads=256 ● Blocks=1 ● Threads/Block = 256 ● Tolerance = 0.000010 ● CPU_Result=11377040 ● GPU_RESULT=11376560 ● PASS/FAIL: PASS
N = 1 Block - 1	N = 255	● N=255 ● Threads=256 ● Blocks=1 ● Threads/Block = 256 ● Tolerance = 0.000010 ● CPU_Result=24363040 ● GPU_RESULT=24362000 ● PASS/FAIL: PASS
N = 1 Block	N = 256	● N=256 ● Threads=256 ● Blocks=1 ● Threads/Block = 256 ● Tolerance = 0.000010 ● CPU_Result=5443616

		• GPU_RESULT=5442576 • PASS/FAIL: PASS
N = 1 Block + 1	N = 257	• N=257 • Threads=512 • Blocks=2 • Threads/Block = 256 • Tolerance = 0.000010 • CPU_Result=29982752 • GPU_RESULT=29981712 • PASS/FAIL: PASS
N not divisible by Block Size	N = 1011	• N=1011 • Threads=1024 • Blocks=4 • Threads/Block = 256 • Tolerance = 0.000010 • CPU_Result=30667664 • GPU_RESULT=30663600 • PASS/FAIL: PASS
Large N for Multiple Blocks	N = 190000	• N=190000 • Threads=190208 • Blocks=743 • Threads/Block = 256 • Tolerance = 0.000010 • CPU_Result=643637264 • GPU_RESULT=644399120 • PASS/FAIL: PASS

AI Disclosure and Miscellaneous Questions and Problems

Disclosure: I generated none of the code from this lab with AI because it was all in the slides. However, I used Claude Sonnet 5 to explain file structure and debug my access into HiPerGator. For instance, I did not realize that I had to move into the source node to use nvcc until I asked Claude. Additionally, I modified some build configurations to make it easier to paste the outputs into the project document. I also used Claude to help me understand how timing works in this project and how to properly use the std::chrono library. This is a fair use since this just speeds up the search process for me.

AI Generated Files:

- node_nav.sh - easier to move into the node
- ./vector_add 1011 | tee "results/n1011.txt" (learned this command from Claude)

Miscellaneous Questions: Integer division will get the floor of the number of threads rather than the ceiling, so there will be less threads than required for an input. For instance, in a case where N is 257, and the number of threads per block = 256, then the number of blocks is 257 / 256 which will divide to 1. This will be a small error since now everything won't be compared as there is one thread left over because integer division rounds down.

Problems: Most of the problems I encountered was with accessing cuda and allocating resources. I noticed that the closer I got to the deadline for this project, the less time I had to use the resources which makes sense. Additionally, I forgot to include the threads library when using the vecadd_mt function, so it threw a few errors. Otherwise, this lab was pretty easy to do.